

# Architecture Report for a Cross-Platform Open-Source Desktop Agent

## Executive summary

The best path is to build **one product with two interaction profiles**—a **Code workspace** for repository work and a **Work assistant** for repetitive tasks, browser actions, research, and deliverable generation—on top of a single local agent platform. The recommended foundation is a **Tauri desktop shell**, a **Rust agent daemon**, and a **React UI**, because Tauri uses the system webview for smaller bundles while Electron embeds Chromium and Node.js, and Rust offers stronger memory-safety and concurrency guarantees than a Node-based long-running daemon. <sup>1</sup>

The product should combine the strongest ideas from current tools without inheriting their weaknesses. From **OpenAI Codex**, borrow worktrees, diff-first review, repository-aware workflows, AGENTS.md, and scheduled/background task patterns. From **Claude Code**, borrow **Plan → Execute** UX, explicit permission modes, hooks, plugins, MCP integration, and the separation between planning and action. From **Hermes**, borrow the shared-core/multi-interface model, durable memory, skill packaging, and SQLite FTS5-based session search. From **OpenHands**, borrow sandbox-first execution and ACP/MCP-style backend composability. From **OpenCode**, **Cline**, **Grok Build**, **Amp**, and **Open Design**, borrow TUI/desktop coexistence, Plan/Act approval patterns, session continuity, AGENTS.md layering, plugin-based extensibility, and local-first artifact workflows. <sup>2</sup>

The most important architectural choice is **storage discipline**. Codex's 2026 SSD-write issues are a concrete warning against uncontrolled local persistence: one class of bugs caused extremely heavy SQLite/WAL write churn, and another repeatedly rewrote tiny JSON files often enough to create multi-GB/day local writes. That means the app must enforce **hash-before-write**, **write batching**, **bounded append logs**, **no timestamp-only rewrites**, and **incremental indexing only for changed files**. SQLite with FTS5 is the best default store for sessions, metadata, approvals, and search; LMDB is a strong optional companion for hot graph key-value state; RocksDB is the least attractive first-choice here because LSM compaction increases write amplification. <sup>3</sup>

Legally, the safest reuse path is to **fork or study Apache-2.0 Codex**, not leaked Claude Code. The OpenAI Codex repository is explicitly Apache-2.0 licensed. Anthropic's own documentation says third-party developers should use **API key authentication**, not claude.ai subscription OAuth, and the April 2026 Claude Code source exposure was an accidental packaging mistake, followed by takedown activity—not an open-source release. <sup>4</sup>

A realistic outcome is a product that starts with a narrow, excellent MVP: **open a repo, plan work, run bounded tasks in an isolated worktree or sandbox, show diffs/tests/resource usage, resume sessions, and automate a few repetitive workflows**. If kept disciplined, this can become remarkable precisely because it does **less by default, better, and with much lower disk/CPU/RAM cost** than many current agent tools. <sup>5</sup>

| Decision area        | Recommendation                                     | Why it wins                                                                                                                              |
|----------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Desktop shell        | <b>Tauri</b>                                       | Smaller bundles via native webview; strong cross-platform story; better chance of meeting install-size goals than Electron. <sup>6</sup> |
| Long-running backend | <b>Rust daemon</b>                                 | Better fit for a resource-capped, always-on orchestrator than Node; safer concurrency than manual shared-state patterns. <sup>7</sup>    |
| Primary local DB     | <b>SQLite + FTS5 + WAL</b>                         | Single-file deployment, native full-text search, proven embedded footprint, good fit for local sessions/metadata. <sup>8</sup>           |
| Secondary graph KV   | <b>LMDB only if needed</b>                         | Strong read latency and copy-on-write, no compaction churn; useful if TokenGraph outgrows SQLite-hot paths. <sup>9</sup>                 |
| Repository isolation | <b>Git worktrees first, containers second</b>      | Worktrees are light and ideal for parallel repo tasks; containers add security for command execution. <sup>10</sup>                      |
| Legal starting point | <b>Reuse/study Codex, avoid leaked Claude Code</b> | Apache-2.0 Codex is reusable; Claude Code leak was accidental and not a license grant. <sup>11</sup>                                     |

## What the reference tools reveal

The strongest products converge on a few design truths. **Codex** makes repository work legible by exposing worktrees, diffs, Git actions, AGENTS.md guidance, and scheduled tasks that can run in the project directory or an isolated worktree. The new ChatGPT desktop app also separates **Chat**, **Work**, and **Codex** as distinct experiences, which is a strong signal that coding and non-coding task work should share infrastructure but not necessarily the same tool palette or default UI. <sup>12</sup>

**Claude Code** is strongest where it makes autonomy inspectable. Its desktop and quickstart docs emphasize **Plan**, **Manual**, and **Accept edits** modes; the broader documentation adds explicit permission modes, hooks, plugins, MCP servers, and scoped settings. That combination is more important than Claude itself: it shows that users trust agents more when the system makes planning, approval, and tool governance visible and editable. Claude Fast is not an official Anthropic source, but as a third-party design reference it is useful because it emphasizes session continuity, orchestration, task routing, and a task system derived from plans—good product ideas, even if not a product source of truth. <sup>13</sup>

**Hermes** is the clearest proof that memory and skills should be treated as first-class local product features rather than bolted-on prompts. Hermes uses the same agent core across desktop, CLI, and gateway; stores sessions in SQLite with FTS5 search; keeps per-profile isolation; distinguishes memory from skills; and supports a learning loop plus scheduled automations and isolated subagents. At the same time, Hermes also exposes what to avoid: broad dependency footprints, bundled runtimes, and context inflation from broadcasting too many skill descriptions into every prompt. <sup>14</sup>

The wider ecosystem fills in the rest of the pattern. **OpenHands** contributes sandboxed execution and ACP-compatible composition. **OpenCode** shows the appeal of one agent across TUI, desktop, IDE, and web, but

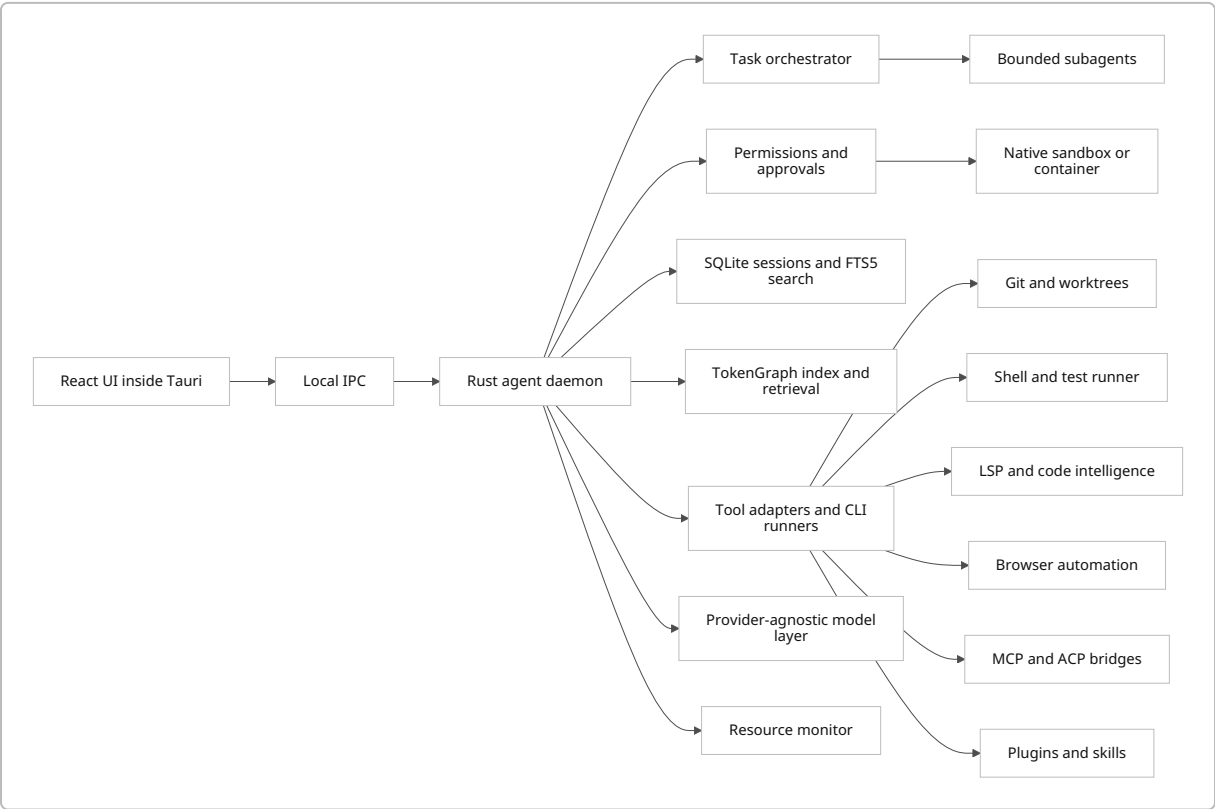
its Windows docs still recommend WSL for the best experience, which is exactly the kind of hidden platform asymmetry a new cross-platform product should avoid. **Cline** shows a solid Plan/Act pattern, approval-first execution, an embeddable SDK, and scheduling hooks. **Amp** reinforces AGENTS.md layering and file-mention driven context. **Grok Build** shows the value of TUI-first extensibility with plugins, hooks, skills, and ACP, but its automatic on-disk session snapshots are also a warning against unbounded local persistence. **Open Design** is valuable because it shows how a local-first desktop app can turn a coding agent into a broader artifact engine via skills and structured project instructions, though its current official desktop emphasis is macOS and Windows rather than a full three-OS story. **BLACKBOX AI** shows how easy it is to over-expand the surface area into IDE, CLI, API, mobile, and multi-agent services all at once. <sup>15</sup>

A final lesson is negative, but crucial: **resource behavior is now a product feature**. Codex users reported sustained SQLite/WAL churn and repeated whole-file rewrites causing enormous disk writes on Windows and macOS. That means “agent memory,” “session history,” “telemetry,” “model caches,” and “review data” must be designed under explicit write budgets from the start, not patched afterward. <sup>16</sup>

| Reference    | Best idea to borrow                                                                                          | What to avoid                                                                                                                             |
|--------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| OpenAI Codex | Worktrees, diff pane, Git controls, AGENTS.md, scheduled tasks. <sup>17</sup>                                | Local storage bugs that create SSD churn; do not mirror that persistence model. <sup>18</sup>                                             |
| ChatGPT Work | A separate “Work” experience for document/research/task workflows in the same app. <sup>19</sup>             | Blurring Code and Work into one overloaded screen by default. This is a product inference based on the official mode split. <sup>19</sup> |
| Claude Code  | Plan/Manual/Accept-edits modes, permission modes, hooks, plugin/MCP stack. <sup>20</sup>                     | Any temptation to reuse leaked source or claude.ai subscription auth in third-party tooling. <sup>21</sup>                                |
| Claude Fast  | Routing, session continuity, plan-to-task conversion. <sup>22</sup>                                          | Treating a third-party workflow kit as an authoritative source for product internals.                                                     |
| Hermes       | Shared core across interfaces; SQLite FTS5 memory; memory vs skills split; isolated subagents. <sup>23</sup> | Heavy dependency/install surface and skill-broadcast context bloat. <sup>24</sup>                                                         |
| OpenHands    | Sandboxed runtime; ACP backend composition; server/UI split. <sup>25</sup>                                   | Pulling in cloud-license pieces that are not open source; keep attention on MIT-licensed core/docs. <sup>26</sup>                         |
| OpenCode     | One agent across terminal, desktop, web, IDE; LSP and multi-session ideas. <sup>27</sup>                     | WSL-first Windows reality if you promise native Windows parity. <sup>28</sup>                                                             |
| Cline        | Plan/Act, approvals, SDK, automation triggers. <sup>29</sup>                                                 | Large plugin and auth scope too early in the roadmap.                                                                                     |
| Grok Build   | ACP/plugins/hooks/skills/TUI extensibility. <sup>30</sup>                                                    | Automatic storage of full prompts/tool calls/file snapshots without strict retention ceilings. <sup>31</sup>                              |

# Recommended architecture

The architecture should be **local-first**, **provider-agnostic**, and **task-profile aware**. The most defensible composition is:



The **desktop shell choice** should be Tauri rather than Electron for this project. Tauri’s official docs emphasize a secure foundation, smaller bundles via the system webview, and cross-platform delivery; Tauri’s v1/v2 materials repeatedly highlight minimal size, even down to sub-megabyte examples in simple cases. Electron remains viable, but its own docs make clear that it embeds Chromium and Node.js and inherits Chromium’s multi-process browser architecture, which is harder to reconcile with the user’s explicit goals around install size and idle resource floor. A native-UI alternative like Qt is mature and cross-platform, but for this product it would likely increase development cost and reduce frontend velocity without enough upside over Tauri + React. <sup>32</sup>

| Desktop stack | Strengths                                                                                                                 | Weaknesses                                                                                          | Recommendation |
|---------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------|
| Tauri + React | Native webview, small bundles, strong Rust integration, good fit for local IPC and privileged backend work. <sup>33</sup> | Webview differences across OSes still need testing; some plugins can add bundle size. <sup>34</sup> | Best default   |

| Desktop stack           | Strengths                                                                       | Weaknesses                                                                                                | Recommendation                         |
|-------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------|
| <b>Electron + React</b> | Mature ecosystem, familiar packaging, many examples. <sup>35</sup>              | Chromium + Node embedded; larger baseline footprint and broader security surface to harden. <sup>36</sup> | Good fallback, not ideal for this goal |
| <b>Qt/QML native</b>    | Mature native-style desktop framework across Windows/macOS/Linux. <sup>37</sup> | Higher UI engineering cost, lower web-stack leverage, more licensing/packaging complexity. <sup>38</sup>  | Strong but less pragmatic here         |

The **backend daemon** should be Rust. Rust's safe subset provides strong memory-safety guarantees and "fearless concurrency," which is a meaningful advantage for a process that will watch files, manage subprocesses, stream tokens, track permissions, and remain alive for long periods. Go is the best fallback language because goroutines and pipelines make concurrent I/O orchestration easy. Node is suitable for adapters and the web UI toolchain, but its official guidance explicitly warns not to block the event loop, which becomes a poor fit when the daemon also needs CPU-bound parsing, search, indexing, and multiple subprocess coordinators. <sup>39</sup>

| Daemon language | What it is good at                                                               | What it is bad at                                                                                               | Recommendation                                  |
|-----------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <b>Rust</b>     | Safe systems programming, concurrency, low overhead. <sup>40</sup>               | Steeper ramp; longer build/debug cycles than Node.                                                              | <b>Best fit</b>                                 |
| <b>Go</b>       | Fast iteration, goroutines, strong I/O orchestration. <sup>41</sup>              | Less control over fine-grained memory/perf than Rust.                                                           | Good second choice                              |
| <b>Node.js</b>  | Great for extensions, adapters, browser tooling, UI-adjacent code. <sup>42</sup> | Event-loop sensitivity, weaker fit for CPU-heavy indexing and strict long-lived resource control. <sup>43</sup> | Use for plugins/helpers, not the primary daemon |

The **provider-agnostic model layer** should not be "one API wrapper." It should be a **capability matrix** over providers. At minimum, every model adapter should declare support for: streaming text, tool calling, structured output, embeddings, image input, context window, reasoning budget knobs, and cost/token accounting. Anthropic's and OpenAI's current ecosystems both emphasize agentic/coding workflows, and Anthropic explicitly requires API-key-based third-party access patterns for developers. That argues for **BYOK** from day one, with first-class adapters for OpenAI, Anthropic, xAI, Google, OpenRouter, Ollama, and any OpenAI-compatible endpoint, but without assuming any provider's subscription login can be re-used in a third-party desktop app. <sup>44</sup>

The **TokenGraph layer** should do more than retrieval. It should be the product’s identity. The right design is a hybrid repository intelligence system that combines: lexical search, AST/symbol indexing, import/call/reference edges, git-history signals, test/build artifacts, and optional embeddings. Hermes demonstrates that **SQLite FTS5 session search** can be both simple and useful. Amp’s AGENTS.md and OpenAI’s AGENTS.md show that structured repo guidance should be layered, discoverable, and path-sensitive. The result should be a retrieval compiler that answers a question like “fix auth race in checkout flow” by combining: the current diff, nearby touched files, symbol neighborhood, build/test failures, relevant AGENTS/skill files, and semantically similar prior sessions. <sup>45</sup>

The **incremental indexing design** should be event-driven and bounded. Do not reindex whole repositories after every edit. Maintain a dirty-file queue fed by file watchers and Git events, parse only changed files, invalidate only affected graph nodes, and update background indexes with debouncing and backpressure. Worktrees should have their own overlay indexes keyed to a shared immutable base so that parallel chats do not duplicate the full repo graph. This is a product inference, but it directly follows from the resource goals and from Codex’s worktree model plus the desire to avoid the kind of repeated local write churn documented in Codex issues. <sup>46</sup>

The **multi-CLI orchestration layer** should treat CLIs as sandboxed tool backends, not as ad hoc shell commands. OpenHands’ ACP backend model is the right inspiration: adapters can spawn an ACP/MCP server or a tightly controlled CLI, then speak a typed protocol over stdio or sockets. For browser workflows, Playwright’s official agent CLI is especially interesting because it already emphasizes a daemon architecture, concise output, isolated sessions, and cross-browser support. That is the right pattern for a “Work” assistant that needs browser automation without flooding the model context or launching a heavyweight browser from scratch on every task step. <sup>47</sup>

The **plugin/skill system** should package instructions, tools, schemas, and optional binaries together, but expose them progressively. Both OpenAI and Claude describe skills/plugins as self-contained capability bundles; Hermes’ memory-vs-skills guidance is especially useful here. The product should therefore load only **skill metadata** into the default prompt, retrieve full skill bodies only when selected, and allow plugins to register MCP servers, hooks, browser recipes, repo rules, schedulers, and artifact generators as separately permissioned capabilities. <sup>48</sup>

A good default UI is a four-quadrant layout:

|                                                                  |                    |                       |
|------------------------------------------------------------------|--------------------|-----------------------|
| Projects / Sessions                                              | Plan and Assistant | Files / Diff / Review |
| Repo tree                                                        | Goal               | Current file          |
| Worktrees                                                        | Plan               | Proposed diff         |
| Tasks                                                            | Tool timeline      | Inline comments       |
| Skills                                                           | Approvals          | Git actions           |
| Schedules                                                        | Subagents          | Test output           |
| Resource Monitor: CPU   RAM   Disk Writes   Cache Size   Workers |                    |                       |

This layout is not copied from any one source, but it is consistent with Codex’s diff/review model, Claude’s plan/approval model, and Hermes’ session/skills orientation. 49

## Storage, performance, and reliability

The safest default storage plan is **SQLite for almost everything user-visible**, with optional LMDB for hot graph state. SQLite gives you local durability, easy backup/export, FTS5, and WAL mode; Hermes already validates SQLite + FTS5 as a practical local memory/session store. LMDB is attractive if TokenGraph’s internal adjacency lists or symbol caches become a bottleneck, because LMDB uses a B+tree with copy-on-write and avoids background compaction. RocksDB is powerful, but its own documentation emphasizes write amplification and compaction tradeoffs, which are exactly the wrong default for a desktop AI agent where SSD wear is a first-class constraint. 50

| Store         | Best use in this product                                                               | Why                                                                 | Caution                                                                                    |
|---------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| SQLite + FTS5 | Sessions, approvals, schedules, cached retrieval metadata, skills registry, search. 51 | Single-file embedded DB; native FTS5; easy shipping and backups. 52 | WAL/checkpoint policy must be tuned to avoid churn. 53                                     |
| LMDB          | Hot graph state, symbol adjacency, immutable-ish KV layers. 9                          | Copy-on-write B+tree, strong reads, no compaction. 9                | Single-writer model complicates highly concurrent writes. 54                               |
| RocksDB       | Probably unnecessary for v1 desktop                                                    | Very high write throughput in server cases. 55                      | LSM compaction implies write amplification; poor fit for SSD-sensitive client software. 55 |

The storage layout should be explicit:

- `state.db` for sessions, approvals, tasks, skills metadata, schedules, and searchable summaries.
- `graph.db` or `graph.lmdb` for TokenGraph nodes, edges, file hashes, symbol spans, and dependency neighborhoods.
- `cache/` for bounded ephemeral artifacts only: test output fragments, retrieved snippets, browser screenshots, temporary diffs.
- `logs/` as append-only, size-capped, rotated event files—not mutable JSON status blobs.

The **SSD-safe write rules** should be non-negotiable. Codex issue reports show two failure modes worth engineering against from day one: rapid SQLite WAL churn driven by logging/feedback writes, and repeated whole-file rewrites of small JSON files that created disproportionate hardware writes. The product should therefore use: **content-hash write suppression, in-memory debounce for frequent state, append logs rather than rewrite snapshots, periodic coalesced commits, fixed retention ceilings, and separate diagnostic verbosity from durable state**. No timestamp or TTL renewal should trigger a file rewrite unless the payload actually changed. 56

The **install-size and runtime budgets** below are recommendations, not observed facts, but they are grounded in the Tauri/Electron tradeoff, the Codex write-bug lessons, and the user’s requirement that the

app remain light on SSD/HDD, CPU, and RAM. Tauri’s small-bundle promise makes these targets aggressive but achievable if browser runtimes, LSP servers, Playwright binaries, and language packagers are kept **lazy-install**. <sup>57</sup>

| Budget target                                             | Recommended ceiling                   |
|-----------------------------------------------------------|---------------------------------------|
| Base installer, Windows/macOS                             | $\leq 50$ MB                          |
| Base package, Linux <code>.deb</code> / <code>.rpm</code> | $\leq 40$ MB                          |
| Base installed footprint                                  | $\leq 120$ MB                         |
| Cold idle RAM after warm-up                               | $\leq 220$ MB                         |
| Typical active RAM during one local task                  | $\leq 650$ MB excluding spawned tools |
| Idle CPU average over 5 minutes                           | $< 1\%$ of one core                   |
| Idle disk writes after warm-up                            | $< 1$ MB/hour                         |
| Default indexer workers                                   | 2                                     |
| Logs retained on disk                                     | $\leq 20$ MB by default               |
| Temporary artifacts retained                              | $\leq 1$ GB, auto-pruned              |

The **resource monitor** should not be an afterthought. It should be visible in the main UI and expose: current CPU, RSS, disk writes per hour, DB file sizes, pending dirty-file queue, active worktrees, active subprocesses, last checkpoint time, and browser/runtime binaries installed. This is partly UX, partly accountability: the product can turn “lightweight” from a marketing adjective into an observable contract. The lack of such observability is one reason users discovered Codex write problems only after checking local storage tools. <sup>58</sup>

The **automated regression suite** should enforce those budgets. These tests matter as much as unit tests.

| Automated test                    | Pass condition                                                                                                                                   |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Idle-write soak test              | After 30-minute warm-up and 2-hour idle, total writes remain below target and no file shows repeated whole-file rewrites without content change. |
| SQLite churn test                 | Under streaming chat and no code edits, WAL growth remains bounded and checkpoints are coalesced.                                                |
| Dirty-file incremental index test | Editing one file updates only that file and impacted graph edges, not a full repo rescan.                                                        |
| Worktree parallelism test         | Two concurrent tasks in separate worktrees do not mutate each other’s checkout or corrupt shared cache state.                                    |
| Plugin install/uninstall test     | Optional features add/remove cleanly without orphaned writes or background loops.                                                                |



| Automated test                  | Pass condition                                                                                            |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| Browser daemon persistence test | Browser automation reuses a daemon/session process and does not relaunch a full browser on every command. |
| Crash-recovery test             | After forced kill, DB reopens cleanly, temp files are pruned, and no replay amplifies writes.             |
| Native Windows/macOS/Linux test | All resource targets are validated on each OS; no platform silently falls back to WSL or a debug logger.  |

## Security, privacy, and legal boundaries

The **permission model** should be explicit, layered, and mode-based. Claude Code's permission and hook model is the clearest public reference here, and Codex's deny-read rules and diff/review patterns reinforce the same lesson: safe agents do not simply ask "allow?" per command; they combine policy, mode, environmental scope, and review affordances. A sensible v1 model is: **Read-only**, **Workspace write**, **Networked task**, and **Elevated/native automation**, with each tool call evaluated against path scope, repo trust, network destination, secret classification, and whether the call occurs in Code mode or Work mode.

59

The **isolation strategy** should be tiered rather than ideological. Use **Git worktrees** for cheap parallel repository task isolation. Use **containers** for code execution and package installs when practical. Use **native process sandboxes** for host-integrated tools and OS automation where containers are awkward. Official platform docs support this layered view: OpenHands recommends Docker sandboxes for local isolation; macOS App Sandbox is a kernel-enforced access-control system; Windows AppContainer and Windows Sandbox provide stronger isolation primitives; and bubblewrap offers lightweight filesystem/process isolation on Linux.

60

A practical isolation matrix looks like this:

| Task type                                   | Preferred isolation                                      |
|---------------------------------------------|----------------------------------------------------------|
| Read/search repository                      | Native process with repo-scoped allowlist                |
| Edit files and run tests in a repo          | Git worktree + optional container                        |
| Install dependencies / run unknown commands | Container by default                                     |
| Browser automation on trusted local sites   | Browser daemon with domain allowlist and session sandbox |
| System automation on the host               | Native sandbox + high-friction approval                  |
| Scheduled background jobs                   | Dedicated worktree and reduced permission profile        |

The **telemetry and privacy defaults** should be conservative. Anthropic's docs around data retention and Claude Code troubleshooting show why users become wary of local agent tools: session transcripts, logs, or source content may be uploaded in some workflows if not clearly bounded. For this product, the default

should be: **no content telemetry, no prompt logging to cloud, crash reports off by default, redaction of known key/token patterns in local diagnostics**, and **explicit user export/share actions for any support bundle**. Privacy-sensitive environments should also be able to disable internet access globally for agents and run only local providers. <sup>61</sup>

The **legal/licensing posture** should be simple:

- **Safe to reuse:** OpenAI Codex repository and any component clearly licensed under Apache-2.0. <sup>62</sup>
- **Potentially safe with review:** MIT-licensed OpenHands core/docs, OpenHands CLI, and other permissively licensed repos such as Playwright or bubblewrap, subject to their actual repository licenses. <sup>63</sup>
- **Not safe to reuse:** leaked Claude Code source or derivative rewrites based on the leaked proprietary code. The April 2026 exposure was an accidental source-map leak, not an open-source release, and Anthropic's public/legal posture afterward was protective, not permissive. <sup>64</sup>
- **Authentication rule:** for Anthropic integrations, third-party developers should use **API keys**, not claude.ai subscription OAuth. <sup>65</sup>

The design implication is straightforward: if you want Claude support, **support Anthropic API keys cleanly**. If you want Codex/OpenAI support, either support OpenAI APIs or any officially documented first-party auth flow as it exists at build time—but do not design around imitation of consumer subscription entitlements unless that is clearly allowed by official documentation. Anthropic's public docs are unusually explicit here, which makes your compliance path clearer than many teams assume. <sup>65</sup>

## Roadmap, effort, and study list

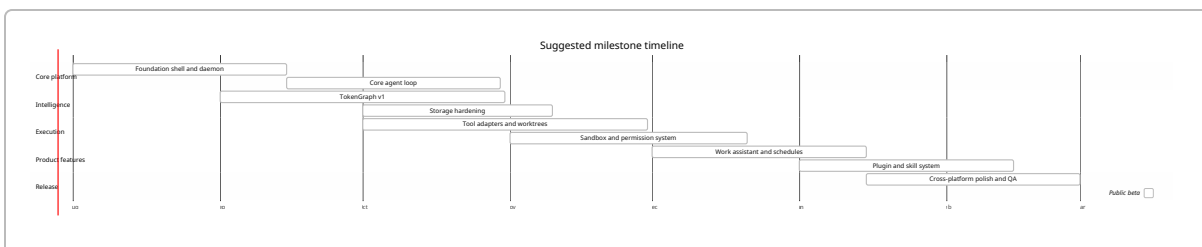
The recommended **MVP** is intentionally narrow. It should be excellent at six things:

- Open a local Git repository and display repo structure, worktrees, active tasks, and diffs.
- Support **Plan → Execute → Review** workflows with approvals and inline diff feedback.
- Run bounded commands, tests, and browser steps through isolated tool adapters.
- Persist sessions, tasks, and searchable history locally with SQLite FTS5.
- Use TokenGraph to retrieve only relevant files/symbol neighborhoods instead of rereading whole repos.
- Schedule simple repetitive tasks such as daily repo health checks, code reviews, or browser-based status collection in dedicated worktrees.

That MVP already covers the core Codex/Work/Claude/Hermes synthesis without overreaching into full autonomous cloud agents, mobile clients, voice-first workflows, or an IDE replacement. It is also aligned with the official patterns surfaced by Codex scheduled tasks/worktrees, Claude plan/permission modes, and Hermes automations and memory. <sup>66</sup>

The **implementation roadmap** below is expressed in **person-months**, so it remains valid even if the team is not yet fixed.

| Milestone               | Deliverable                                                                             | Estimated effort |
|-------------------------|-----------------------------------------------------------------------------------------|------------------|
| Foundation              | Tauri shell, Rust daemon, React workspace UI, local IPC, minimal settings, repo opening | 3–4 PM           |
| Core agent loop         | Provider-agnostic model adapters, task state machine, approvals, diff/review surface    | 4–6 PM           |
| TokenGraph v1           | Incremental parsing, symbol graph, changed-file invalidation, retrieval compiler        | 4–6 PM           |
| Execution layer         | Shell/git/LSP adapters, worktrees, browser daemon integration, MCP/ACP bridge           | 5–7 PM           |
| Storage hardening       | SQLite schemas, FTS5 search, write batching, crash recovery, resource monitor           | 3–4 PM           |
| Sandboxing and security | Container runner, native sandbox fallbacks, secret redaction, permission policies       | 4–6 PM           |
| Work assistant          | Scheduled repetitive tasks, browser flows, artifact templates, report pipeline          | 3–5 PM           |
| Plugin/skill ecosystem  | Skill manifests, lazy installs, signed bundles, plugin runtime policies                 | 3–5 PM           |
| Cross-platform polish   | Native Windows/macOS/Linux QA, installers, notarization/signing, perf tuning            | 4–6 PM           |



The **highest-value open-source components to fork or study** are the ones that give you architecture, not just UI inspiration. The list below is ordered roughly by expected value to your build. The URLs are included directly because you explicitly asked for them.

OpenAI Codex repo

<https://github.com/openai/codex>

OpenAI Codex docs

<https://developers.openai.com/codex/cli>

<https://developers.openai.com/codex/automations>

<https://developers.openai.com/codex/environments/git-worktrees>

<https://developers.openai.com/codex/agent-configuration/agents-md>

OpenAI ChatGPT Work and Codex help

<https://help.openai.com/en/articles/20001275-chatgpt-work-and-codex>

<https://help.openai.com/en/articles/20001276-moving-to-the-new-chatgpt-desktop-app>

Claude Code official docs

<https://code.claude.com/docs/en/overview>

<https://code.claude.com/docs/en/desktop>

<https://code.claude.com/docs/en/permissions>

<https://code.claude.com/docs/en/hooks>

<https://code.claude.com/docs/en/plugins-reference>

<https://code.claude.com/docs/en/mcp>

Anthropic official legal/compliance docs

<https://docs.anthropic.com/en/docs/claude-code/legal-and-compliance>

<https://docs.anthropic.com/en/docs/claude-code/sdk>

Claude Fast

<https://claudefa.st/>

<https://claudefa.st/readme>

Hermes Agent

<https://github.com/nousresearch/hermes-agent>

<https://hermes-agent.nousresearch.com/docs/>

<https://hermes-agent.nousresearch.com/docs/user-guide/desktop>

<https://hermes-agent.nousresearch.com/docs/user-guide/features/memory>

<https://hermes-agent.nousresearch.com/docs/user-guide/sessions>

OpenHands

<https://github.com/OpenHands/openhands>

<https://docs.openhands.dev/>

<https://docs.openhands.dev/openhands/usage/sandboxes/overview>

<https://docs.openhands.dev/sdk>

OpenCode

<https://github.com/opencode-ai/opencode>

<https://opencode.ai/docs/>

<https://opencode.ai/download>

Cline

<https://github.com/cline/cline>

<https://cline.bot/>

Amp

<https://ampcode.com/>

<https://ampcode.com/manual>

Grok Build  
<https://github.com/xai-org/grok-build>  
<https://docs.x.ai/build/overview>

Open Design  
<https://github.com/nexu-io/open-design>  
<https://open-design.ai/>

BLACKBOX AI  
<https://www.blackbox.ai/>  
<https://www.blackbox.ai/cli>

Playwright  
<https://playwright.dev/>  
<https://playwright.dev/agent-cli/introduction>

Bubblewrap  
<https://github.com/containers/bubblewrap>

Tauri  
<https://v2.tauri.app/>  
<https://github.com/tauri-apps/tauri>

The **risk analysis** is manageable, but only if handled up front.

| Risk                                  | Why it matters                                                                                            | Mitigation                                                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Legal contamination</b>            | Studying or deriving from leaked Claude Code could compromise the project's defensibility. <sup>64</sup>  | Use Apache-2.0 Codex and permissive OSS references only; document clean-room boundaries.               |
| <b>Platform asymmetry</b>             | Several current tools are effectively better on macOS or WSL than on native Windows/Linux. <sup>67</sup>  | Run native CI and perf suites on all three OSes from the first milestone.                              |
| <b>SSD wear / write amplification</b> | Codex 2026 issues show a real, user-visible failure mode. <sup>18</sup>                                   | Add write-budget tests, hash-before-write, append logs, retention ceilings, and runtime write metrics. |
| <b>Security drift</b>                 | Agents that mix repo access, browser use, and host automation can become dangerous quickly. <sup>68</sup> | Tiered permissions, worktrees, per-tool sandboxes, secret redaction, network allowlists.               |
| <b>Scope explosion</b>                | BLACKBOX/Cline/OpenCode-style breadth is attractive but expensive. <sup>69</sup>                          | Preserve a tight MVP and gate optional features behind lazy-install bundles.                           |

| Risk                          | Why it matters                                                                                      | Mitigation                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Context bloat</b>          | Hermes issue history shows how easily skill metadata balloons prompt cost. <sup>70</sup>            | Metadata-only skill discovery by default; retrieve full skill bodies on demand. |
| <b>Vendor-policy breakage</b> | Anthropic is explicit that third-party tools should use API keys, not consumer OAuth. <sup>65</sup> | Build around BYOK and documented provider APIs only.                            |

The bottom-line recommendation is this: build a **Linux/macOS/Windows Tauri app with a Rust daemon**, make **TokenGraph** the differentiator, adopt **Codex-style worktrees and review**, **Claude-style planning and approvals**, **Hermes-style memory and skills**, and **OpenHands-style sandboxable execution**, then enforce **resource ceilings in CI** as if they were security requirements. If you do that, the result will not just “look like” the best parts of today’s tools; it will likely be **more defensible, lighter, and more durable** than several of them. <sup>71</sup>

<sup>1</sup> <sup>6</sup> <sup>32</sup> <sup>33</sup> <sup>57</sup> <sup>71</sup> What is Tauri?

[https://v2.tauri.app/start?utm\\_source=chatgpt.com](https://v2.tauri.app/start?utm_source=chatgpt.com)

<sup>2</sup> <sup>5</sup> <sup>10</sup> <sup>12</sup> <sup>17</sup> <sup>46</sup> Worktrees | ChatGPT Learn - OpenAI Developers

[https://learn.chatgpt.com/docs/environments/git-worktrees?utm\\_source=chatgpt.com](https://learn.chatgpt.com/docs/environments/git-worktrees?utm_source=chatgpt.com)

<sup>3</sup> <sup>16</sup> <sup>18</sup> <sup>56</sup> <sup>58</sup> Codex SQLite feedback logs can write ~640 TB/year and ...

[https://github.com/openai/codex/issues/28224?utm\\_source=chatgpt.com](https://github.com/openai/codex/issues/28224?utm_source=chatgpt.com)

<sup>4</sup> <sup>11</sup> <sup>62</sup> openai/codex: Lightweight coding agent that runs in your ...

[https://github.com/openai/codex?utm\\_source=chatgpt.com](https://github.com/openai/codex?utm_source=chatgpt.com)

<sup>7</sup> <sup>39</sup> <sup>40</sup> Fearless Concurrency - The Rust Programming Language

[https://doc.rust-lang.org/book/ch16-00-concurrency.html?utm\\_source=chatgpt.com](https://doc.rust-lang.org/book/ch16-00-concurrency.html?utm_source=chatgpt.com)

<sup>8</sup> <sup>50</sup> <sup>51</sup> <sup>52</sup> SQLite FTS5 Extension

[https://www.sqlite.org/fts5.html?utm\\_source=chatgpt.com](https://www.sqlite.org/fts5.html?utm_source=chatgpt.com)

<sup>9</sup> <sup>54</sup> Lightning Memory-Mapped Database Manager (LMDB)

[https://www.lmdb.tech/doc/?utm\\_source=chatgpt.com](https://www.lmdb.tech/doc/?utm_source=chatgpt.com)

<sup>13</sup> <sup>20</sup> Desktop application - Claude Code Docs

[https://code.claude.com/docs/en/desktop?utm\\_source=chatgpt.com](https://code.claude.com/docs/en/desktop?utm_source=chatgpt.com)

<sup>14</sup> <sup>23</sup> Desktop App - Hermes Agent - nous research

[https://hermes-agent.nousresearch.com/docs/user-guide/desktop?utm\\_source=chatgpt.com](https://hermes-agent.nousresearch.com/docs/user-guide/desktop?utm_source=chatgpt.com)

<sup>15</sup> <sup>25</sup> <sup>60</sup> Overview - OpenHands Docs

[https://docs.openhands.dev/openhands/usage/sandboxes/overview?utm\\_source=chatgpt.com](https://docs.openhands.dev/openhands/usage/sandboxes/overview?utm_source=chatgpt.com)

<sup>19</sup> ChatGPT Work and Codex

[https://help.openai.com/en/articles/20001275-chatgpt-work-and-codex?utm\\_source=chatgpt.com](https://help.openai.com/en/articles/20001275-chatgpt-work-and-codex?utm_source=chatgpt.com)

<sup>21</sup> <sup>65</sup> Legal and compliance - Claude Code Docs

[https://docs.anthropic.com/en/docs/claude-code/legal-and-compliance?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/docs/claude-code/legal-and-compliance?utm_source=chatgpt.com)

22 AI Led Development + Growth Marketing Kits - Claude Fast

[https://claudefa.st/readme?utm\\_source=chatgpt.com](https://claudefa.st/readme?utm_source=chatgpt.com)

24 GitHub - NousResearch/hermes-agent: The agent that grows with you · GitHub

<https://github.com/nousresearch/hermes-agent>

26 All Hands AI OpenHands Self-hosted Cloud

[https://github.com/OpenHands/openhands-cloud?utm\\_source=chatgpt.com](https://github.com/OpenHands/openhands-cloud?utm_source=chatgpt.com)

27 OpenCode | The open source AI coding agent

[https://opencode.ai/?utm\\_source=chatgpt.com](https://opencode.ai/?utm_source=chatgpt.com)

28 67 Windows (WSL)

[https://opencode.ai/docs/windows-wsl/?utm\\_source=chatgpt.com](https://opencode.ai/docs/windows-wsl/?utm_source=chatgpt.com)

29 cline/cline: Autonomous coding agent as an SDK ...

[https://github.com/cline/cline?utm\\_source=chatgpt.com](https://github.com/cline/cline?utm_source=chatgpt.com)

30 Introducing Grok Build

[https://x.ai/news/grok-build-cli?utm\\_source=chatgpt.com](https://x.ai/news/grok-build-cli?utm_source=chatgpt.com)

31 Sessions

[https://docs.x.ai/build/features/sessions?utm\\_source=chatgpt.com](https://docs.x.ai/build/features/sessions?utm_source=chatgpt.com)

34 Configuration

[https://v2.tauri.app/reference/config/?utm\\_source=chatgpt.com](https://v2.tauri.app/reference/config/?utm_source=chatgpt.com)

35 36 Electron: Build cross-platform desktop apps with JavaScript ...

[https://electronjs.org/?utm\\_source=chatgpt.com](https://electronjs.org/?utm_source=chatgpt.com)

37 38 Qt Framework – Build Fast, Scalable Cross-Platform Software

[https://www.qt.io/development/qt-framework?utm\\_source=chatgpt.com](https://www.qt.io/development/qt-framework?utm_source=chatgpt.com)

41 Goroutines

[https://go.dev/tour/concurrency?utm\\_source=chatgpt.com](https://go.dev/tour/concurrency?utm_source=chatgpt.com)

42 43 Don't Block the Event Loop (or the Worker Pool)

[https://nodejs.org/learn/asynchronous-work/dont-block-the-event-loop?utm\\_source=chatgpt.com](https://nodejs.org/learn/asynchronous-work/dont-block-the-event-loop?utm_source=chatgpt.com)

44 Agent SDK overview - Claude Code Docs

[https://docs.anthropic.com/en/docs/claude-code/sdk?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/docs/claude-code/sdk?utm_source=chatgpt.com)

45 Persistent Memory | Hermes Agent - nous research

[https://hermes-agent.nousresearch.com/docs/user-guide/features/memory?utm\\_source=chatgpt.com](https://hermes-agent.nousresearch.com/docs/user-guide/features/memory?utm_source=chatgpt.com)

47 ACP Agent

[https://docs.openhands.dev/sdk/guides/agent-acp?utm\\_source=chatgpt.com](https://docs.openhands.dev/sdk/guides/agent-acp?utm_source=chatgpt.com)

48 Build skills | ChatGPT Learn

[https://developers.openai.com/codex/build-skills?utm\\_source=chatgpt.com](https://developers.openai.com/codex/build-skills?utm_source=chatgpt.com)

49 Local environments | ChatGPT Learn

[https://learn.chatgpt.com/docs/environments/local-environment?utm\\_source=chatgpt.com](https://learn.chatgpt.com/docs/environments/local-environment?utm_source=chatgpt.com)

53 Write-Ahead Logging

[https://www.sqlite.org/wal.html?utm\\_source=chatgpt.com](https://www.sqlite.org/wal.html?utm_source=chatgpt.com)

55 **RocksDB Tuning Guide**

[https://github.com/facebook/rocksdb/wiki/RocksDB-Tuning-Guide?utm\\_source=chatgpt.com](https://github.com/facebook/rocksdb/wiki/RocksDB-Tuning-Guide?utm_source=chatgpt.com)

59 **Choose a permission mode - Claude Code Docs**

[https://code.claude.com/docs/en/permission-modes?utm\\_source=chatgpt.com](https://code.claude.com/docs/en/permission-modes?utm_source=chatgpt.com)

61 **API and data retention - Claude Platform Docs**

[https://docs.anthropic.com/en/docs/build-with-claude/zero-data-retention?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/docs/build-with-claude/zero-data-retention?utm_source=chatgpt.com)

63 **OpenHands/CREDITS.md at main**

[https://github.com/OpenHands/OpenHands/blob/main/CREDITS.md?utm\\_source=chatgpt.com](https://github.com/OpenHands/OpenHands/blob/main/CREDITS.md?utm_source=chatgpt.com)

64 **Anthropic Accidentally Exposes Claude Code Source via ...**

[https://www.infoq.com/news/2026/04/claude-code-source-leak/?utm\\_source=chatgpt.com](https://www.infoq.com/news/2026/04/claude-code-source-leak/?utm_source=chatgpt.com)

66 **Scheduled tasks | ChatGPT Learn**

[https://developers.openai.com/codex/automations?utm\\_source=chatgpt.com](https://developers.openai.com/codex/automations?utm_source=chatgpt.com)

68 **Configure permissions - Claude Code Docs**

[https://code.claude.com/docs/en/permissions?utm\\_source=chatgpt.com](https://code.claude.com/docs/en/permissions?utm_source=chatgpt.com)

69 **BLACKBOX AI**

[https://www.blackbox.ai/?utm\\_source=chatgpt.com](https://www.blackbox.ai/?utm_source=chatgpt.com)

70 **Skill list bloat causes massive context window inflation ...**

[https://github.com/NousResearch/hermes-agent/issues/22620?utm\\_source=chatgpt.com](https://github.com/NousResearch/hermes-agent/issues/22620?utm_source=chatgpt.com)